数值线性代数　第 14 讲：稳定性

Lloyd N. Trefethen、David Bau III　中文阅读版

如果数值算法能够为数值问题提供精确解，那当然是很理想的事情. 然而，问题是连续的，而数字计算机是离散的，因此这通常不可能实现. 稳定性是刻画“能够做到什么”的标准方式，也是数值分析家理解“得到正确答案”的方式，即使这个答案并不精确.

算法

在第 12 讲中，我们把数学问题定义为从数据向量空间 X 到解向量空间 Y 的函数

$$f : X \to Y.$$

算法可以看作同两个空间之间的另一个映射 $\widetilde{f} : X \to Y$. 下面给出这一说法的精确定义. 固定一个问题 f、一个浮点系统满足式 (13.7)（但不一定满足式 (13.6)）的计算机、一个求解 f 的算法（这里以宽泛意义使用“算法”一词），以及该算法在计算机程序中的实现.

给定数据 $x \in X$，先按照满足式 (13.5) 的方式把数据舍入为浮点数，再将其输入计算机程序. 运行程序后，得到一组属于向量空间 Y 的浮点数（因为算法的设计目标就是求解 f）. 把这个计算结果记为 $\widetilde{f}(x)$.

这个情形可能非常复杂! 至少，$\widetilde{f}(x)$ 会受到舍入误差的影响. 根据具体情况，它还可能受到收敛容差，甚至计算机上同时运行的其他任务等各种因素的影响；例如，当计算分配给处理器的方式直到运行时才确定时，就会出现这种情况. 因此，所谓的“函数”$\widetilde{f}(x)$ 甚至可能在两次运行之间取不同的值；它可能是多值的.（事实上，问题 f 本身也应允许是多值的，这样就能处理允许非唯一解的情形，例如复数的两个平方根都可以接受.）然而，尽管有这些复杂性，我们仍将看到：只依靠式 (13.5) 和 (13.7) 这两个基本公理，就可以对 $\widetilde{f}(x)$，进而对数值线性代数算法的准确性，作出出人意料地简洁的陈述.

波浪号 $\widetilde{\ }$ 记号非常方便. 正如 $\widetilde{f}$ 是 f 的计算机计算对应物一样，本书中其他计算得到的量也经常用波浪号标记. 例如，方程组 $Ax = b$ 的计算解可以记为 $\widetilde{x}$.

准确性

除非是平凡情形，$\widetilde{f}$ 不可能连续. 不过，一个好的算法应当逼近相应的数学问题 f. 为了定量描述这一点，可以考虑计算的绝对误差

$$\|\widetilde{f}(x) - f(x)\|,$$

或者相对误差

$$\frac{\|\widetilde{f}(x) - f(x)\|}{\|f(x)\|}. \tag{14.1}$$

本书主要使用相对量，因此式 (14.1) 将作为标准误差度量.

如果 $\widetilde{f}$ 是一个好的算法，自然会希望相对误差很小，量级为 $\varepsilon_{\text{machine}}$. 对于每个 $x \in X$，如果算法 $\widetilde{f}$ 满足

$$\frac{\|\widetilde{f}(x) - f(x)\|}{\|f(x)\|} = O(\varepsilon_{\text{machine}}), \tag{14.2}$$

就称它对问题 f 是准确的.

粗略地说，式 (14.2) 中的 $O(\varepsilon_{\text{machine}})$ 表示“机器 epsilon 的量级”. 不过，$O(\varepsilon_{\text{machine}})$ 也有精确定义，我们马上讨论这一点；这也会说明当分母为零时，类似式 (14.2) 的公式应如何理解.

稳定性

如果问题 f 是病态的，那么式 (14.2) 所定义的准确性目标就过于苛刻了. 在数字计算机上，输入数据的舍入不可避免；即使后续所有计算都能完美进行，这一扰动本身也可能导致结果发生显著变化. 因此，一般情况下最适合追求的不是处处准确，而是稳定性.

如果对每个 $x \in X$，都存在某个 $\widetilde{x}$，使得下面两式同时成立：

$$\frac{\|\widetilde{f}(x) - f(\widetilde{x})\|}{\|f(\widetilde{x})\|} = O(\varepsilon_{\text{machine}}), \tag{14.3}$$

$$\frac{\|\widetilde{x} - x\|}{\|x\|} = O(\varepsilon_{\text{machine}}). \tag{14.4}$$

那么称算法 $\widetilde{f}$ 对问题 f 是稳定的.

用一句话说：

稳定算法对一个几乎正确的问题给出几乎正确的答案.

这一讲定义的动机将在下一讲以及本书后面的应用中逐渐清楚. 需要提醒读者的是：这里给出的稳定性定义在数值线性代数的许多部分都很有用，但条件 $O(\varepsilon_{\text{machine}})$ 对微分方程等其他领域的全部数值问题而言可能过于严格.

后向稳定性

许多数值线性代数算法满足一个比稳定性更强、也更简单的条件. 如果对每个 $x \in X$，都有

$$\widetilde{f}(x) = f(\widetilde{x}) \quad \text{对某个} \widetilde{x} \text{ 满足} \quad \frac{\|\widetilde{x} - x\|}{\|x\|} = O(\varepsilon_{\text{machine}}), \tag{14.5}$$

则称算法 $\widetilde{f}$ 对问题 f 是后向稳定的.

这是对稳定性定义的加强：式 (14.3) 中的 $O(\varepsilon_{\text{machine}})$ 被替换成了零. 换句话说：

后向稳定算法对一个几乎正确的问题给出完全正确的答案.

例子将在下一讲给出.

$O(\varepsilon_{\text{machine}})$ 的含义

现在解释式 (14.2)–(14.5) 中"$O(\varepsilon_{\text{machine}})$"的精确含义. 记号

$$\varphi(t) = O(\psi(t)) \tag{14.6}$$

是数学中的标准记号，并且有精确定义. 它断言存在某个正数 C，使得当 t 足够接近某个默认的极限（例如 $t \to 0$ 或 $t \to \infty$）时，

$$|\varphi(t)| \le C\psi(t). \tag{14.7}$$

例如，陈述 $\sin^2 t = O(t^2)$（当 $t \to 0$）意味着存在常数 C，使得对所有充分小的 t，都有 $|\sin^2 t| \le Ct^2$.

数学中还有形如

$$\varphi(s,t) = O(\psi(t)) \quad \text{关于}s\text{ 一致成立} \tag{14.8}$$

的陈述，其中 φ 不仅依赖于 t，还依赖于另一个变量 s."一致"表示式 (14.7) 中的同一个常数 C 对 s 的所有取值都成立. 例如，

$$(\sin^2 t)(\sin^2 s) = O(t^2)$$

在 $t \to 0$ 时关于 s 一致成立；但如果把正弦换成 s^2，这种一致性就会丢失.

本书对符号 O 的使用遵循这些标准定义. 具体地，我们经常陈述如下形式的结果：

$$\|\text{计算得到的量}\| = O(\varepsilon_{\text{machine}}). \tag{14.9}$$

式 (14.9) 的含义如下. 首先，"计算得到的量"表示由问题 f 的算法 $\widetilde{f}$ 所确定的某个数或数的集合的范数，它同时依赖于问题 f 的数据 $x \in X$ 和 $\varepsilon_{\text{machine}}$. 相对误差 (14.1) 就是一个例子. 其次，隐含的极限过程是 $\varepsilon_{\text{machine}} \to 0$（即 $\varepsilon_{\text{machine}}$ 是式 (14.8) 中对应于 t 的变量）. 第三，符号 O 对所有数据 $x \in X$ 一致成立（即 x 是对应于 s 的变量）. 我们很少专门说明关于 $x \in X$ 的一致性，但它总是隐含的.

在任何一台具体的机器上，$\varepsilon_{\text{machine}}$ 都是固定量. 谈论极限 $\varepsilon_{\text{machine}} \to 0$ 时，我们是在考虑计算机，或者更准确地说，计算机族的理想化. 式 (14.9) 的意思是：如果把有关算法放到一系列满足式 (13.5) 和 (13.7)、且 $\varepsilon_{\text{machine}}$ 逐渐减小到零的计算机上运行，那么 $\|$计算得到的量$\|$ 就保证按 $\varepsilon_{\text{machine}}$ 的比例或更快地减小. 这些理想计算机只要求满足式 (13.5) 和 (13.7)，没有其他要求.

依赖于 m, n，而不是 A, b

我们还可以进一步说明式 (14.2)–(14.5) 中 $O(\varepsilon_{\text{machine}})$ 的含义. 隐含常数的一致性可以通过下面的例子说明. 假设我们正在研究一个算法，它对非奇异 $m \times m$ 方程组 $Ax = b$ 求解 x，并且断言该算法的计算结果 $\widetilde{x}$ 满足

$$\frac{\|\widetilde{x} - x\|}{\|x\|} = O\big(\kappa(A)\varepsilon_{\text{machine}}\big). \tag{14.10}$$

这一断言意味着：存在一个与矩阵 A 和右端 b 都无关的单一常数 C，使得对所有充分小的 $\varepsilon_{\text{machine}}$，

$$\frac{\|\widetilde{x} - x\|}{\|x\|} \le C\kappa(A)\varepsilon_{\text{machine}}. \tag{14.11}$$

如果类似式 (14.11) 的公式中分母为零，则采用如下约定. 我们真正想表达的是

$$\|\widetilde{x} - x\| \le C\kappa(A)\varepsilon_{\text{machine}}\|x\|. \tag{14.12}$$

当 $\|x\| \neq 0$ 时，两种写法没有区别；但当 $\|x\| = 0$ 时，式 (14.12) 清楚地表明，式 (14.10) 的精确含义是：对所有充分小的 $\varepsilon_{\text{machine}}$，都有 $\|\widetilde{x} - x\| = 0$.

虽然式 (14.11) 或 (14.12) 中的常数 C 不依赖于 A 或 b，但一般会依赖于维数 m. 从形式上说，这是第 12 讲问题定义的结果. 如果定义问题 f 的维数（例如 m 或 n）发生变化，那么向量空间 X 和 Y 也必须变化，于是我们实际上得到了一个新问题 f'.

从实际角度看，数值线性代数算法中的舍入误差影响通常也会随 m 和 n 增长. 不过，这种增长通常足够慢，不会造成严重问题. 在最坏情况下，对 m 或 n 的依赖通常是线性、二次或三次的（指数取决于所选范数以及所选算法）；而由于统计抵消，大多数数据的误差远小于最坏情况.

原则上，类似式 (14.9) 的陈述可能隐藏一个 2^m 这样的维数相关因子，使界在实践中毫无用处. 然而，本书中只有一个地方会出现这种情况，即带部分选主元的高斯消去法；届时我们会充分提醒读者，避免误解. 一般而言，当本书印出 $O(\varepsilon_{\text{machine}})$ 时，在真实机器上进行真实计算时，相关量通常至多是 $\varepsilon_{\text{machine}}$ 的 100 倍，或者也许 1000 倍.

与范数的独立性

涉及 $O(\varepsilon_{\text{machine}})$ 的定义有一个方便的性质：只要 X 和 Y 是有限维空间，这些定义就与范数的选择无关.

定理 14.1. 对于定义在有限维空间 X 和 Y 上的问题 f 及算法 $\widetilde{f}$，准确性、稳定性和后向稳定性这三个性质是否成立，都独立于 X 和 Y 中范数的选择.

证明. 众所周知（也很容易证明），在有限维向量空间中，所有范数都是等价的. 也就是说，如果 $\|\cdot\|$ 和 $\|\cdot\|'$ 是同一空间上的两个范数，那么存在正数 C_1, C_2，使得对该空间中的所有 x，

$$C_1\|x\| \le \|x\|' \le C_2\|x\|.$$

因此，改变范数可能改变 $O(\varepsilon_{\text{machine}})$ 陈述中隐含常数 C 的大小，但不会改变这样一个常数是否存在. □

习题

14.1 判断正误.

(a) $\sin x = O(1)$，当 $x \to \infty$.

(b) $\sin x = O(1)$，当 $x \to 0$.

(c) $\log x = O(x^{1/100})$，当 $x \to \infty$.

(d) $n! = O((n/e)^n)$，当 $n \to \infty$.

(e) $A = O(V^{2/3})$，当 $V \to \infty$；其中 A 和 V 分别是球的表面积和体积，度量单位分别为平方英里和立方微米.

(f) $\text{fl}(\pi) - \pi = O(\varepsilon_{\text{machine}})$.（这里没有写出极限 $\varepsilon_{\text{machine}} \to 0$，因为本书所有 $O(\varepsilon_{\text{machine}})$ 表达式都默认这一点.）

(g) $\text{fl}(n\pi) - n\pi = O(\varepsilon_{\text{machine}})$，对所有整数 n 一致成立.（这里的 $n\pi$ 表示精确的数学量，而不是浮点计算的结果.）

14.2 (a) 证明

$$(1 + O(\varepsilon_{\text{machine}}))(1 + O(\varepsilon_{\text{machine}})) = 1 + O(\varepsilon_{\text{machine}}).$$

这一陈述的精确含义是：如果函数 f 满足

$$f(\varepsilon_{\text{machine}}) = (1 + O(\varepsilon_{\text{machine}}))(1 + O(\varepsilon_{\text{machine}})) \quad \text{当} \varepsilon_{\text{machine}} \to 0,$$

那么也有

$$f(\varepsilon_{\text{machine}}) = 1 + O(\varepsilon_{\text{machine}}) \quad \text{当} \varepsilon_{\text{machine}} \to 0.$$

(b) 证明

$$(1 + O(\varepsilon_{\text{machine}}))^{-1} = 1 + O(\varepsilon_{\text{machine}}).$$